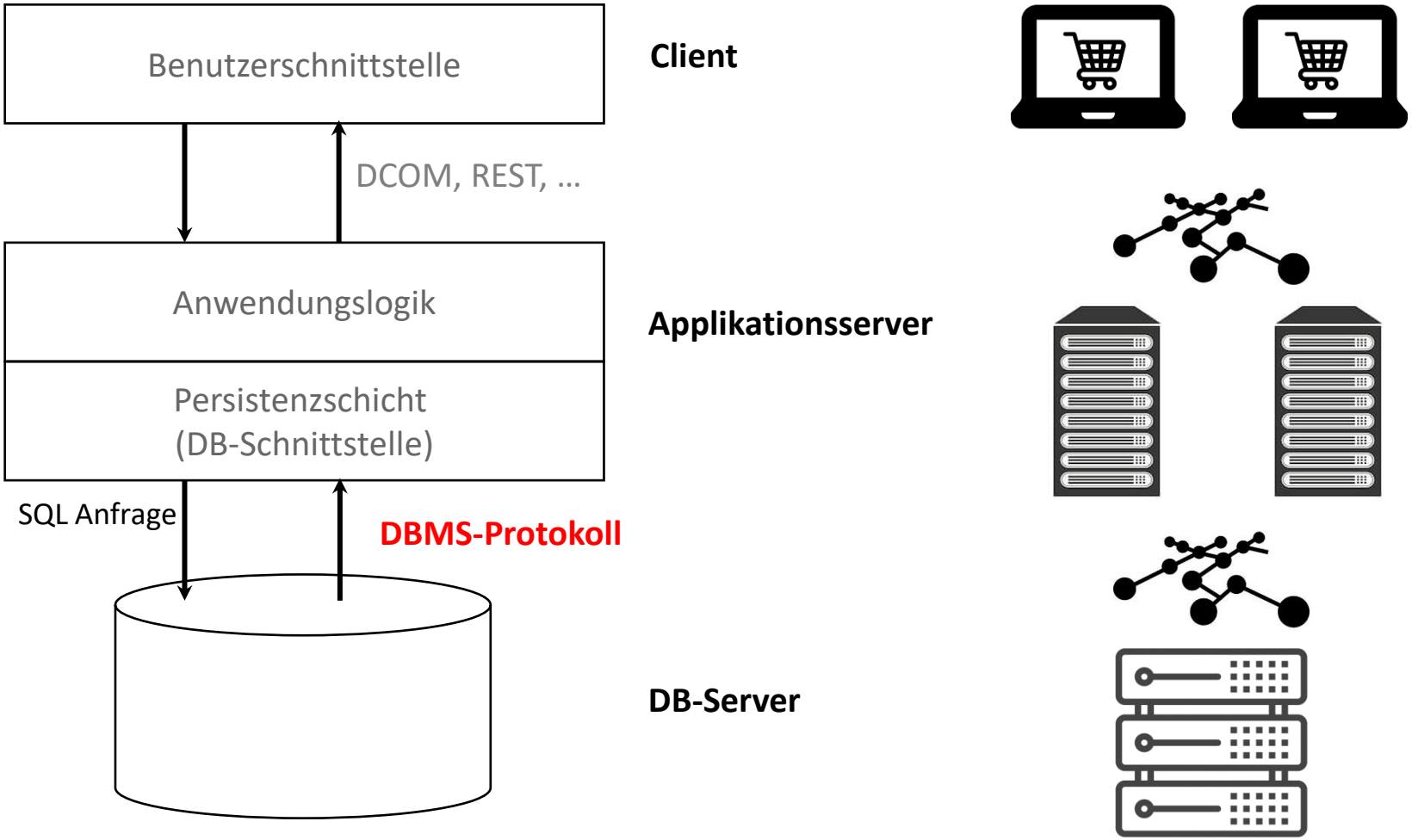
PARIS LODRON UNIVERSITÄT SALZBURG

# Datenbank Tuning

**Scope**

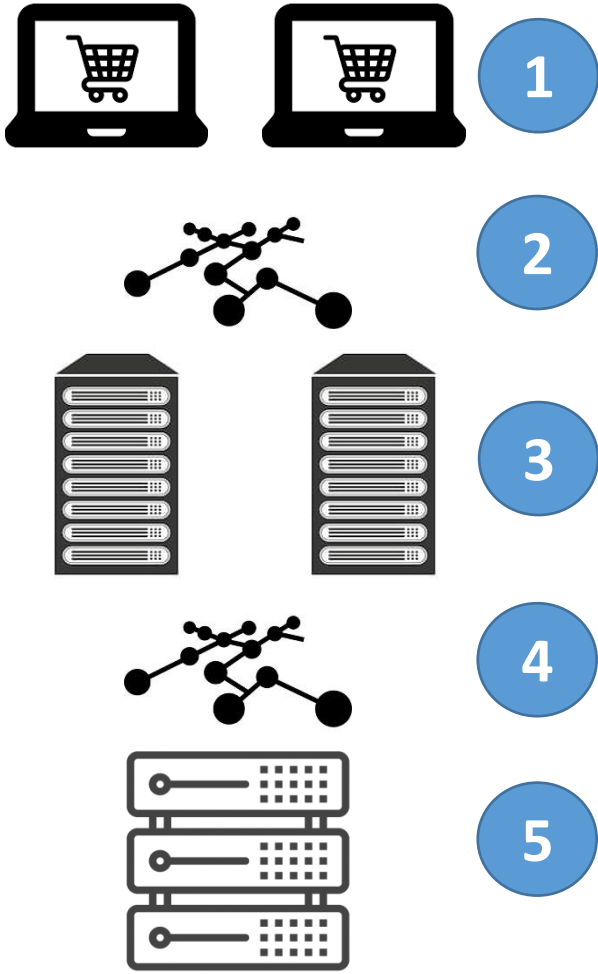
(und Dinge, die wir auch nicht machen, aber relevant sind…)

# Client-Server Architektur: Typischer Aufbau



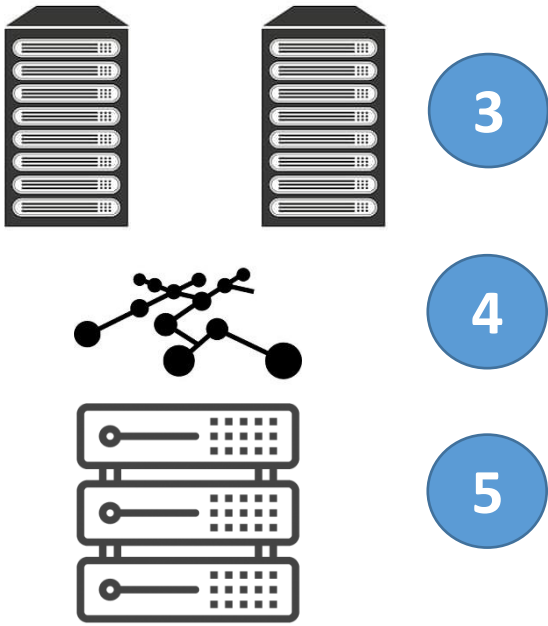
# Client-Server Architektur: Mögliche Problemursachen

1. Client selbst
2. Netzwerk zwischen Client und Applikation (Internet?)
3. Applikationsserver
4. Netzwerk zwischen Applikationsserver und Datenbank
5. Datenbankserver



# Client-Server Architektur: Mögliche Problemursachen

1. Client selbst
2. Netzwerk zwischen Client und Applikation (Internet?)
3. Applikationsserver
4. Netzwerk zwischen Applikationsserver und Datenbank
5. Datenbankserver



# Lokalisation des Problems

PARIS LODRON UNIVERSITÄT SALZBURG

I. Reproduzierbarkeit – Nicht reproduzierbare Problem lassen sich schwer lösen

II. Ausschluss alternativen Ursachen
- Last-Monitoring auf Netzwerk, Applikations- & DB-Server
- Beispiele:
  - Last ist generell hoch
  - DB Server wir auch als Back-Up Server verwendet
  - Andere Applikationen auf App-Server

III. Beginn des eigentlichen Datenbank Tunings

# Prinzipien des DB Tunings

PARIS LODRON UNIVERSITÄT SALZBURG

- Meist sehr lokales Problem in komplexem System
- Tatsächliche Ursache identifizieren

(1) Lokal lösen

- Arzt soll Krankheit heilen ohne Nebenwirkungen heilen, nicht nur Symptome lindern
- DB-Bsp: Festplattenaktivität hoch: Was erzeugt den Aufwand vs. schnellere HDDs kaufen

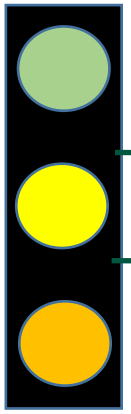
(2) Oder Verschieben

- Vermeiden von Rush Hour auf dem Weg zur Arbeit
- Scheduled task
- Verteilen der Last auf redundantes System

# Der Tuning Prozess

1. SQL Anti-Patterns entfernen
2. Anfragetuning
3. Index Tuning
4. Transaction Tuning
5. Hardware Tuning

# Der Tuning Prozess



1. SQL Anti-Patterns entfernen
2. Anfragetuning

**(Fast) keine Auswirkungen auf DB**

3. Index Tuning

**Geringe Auswirkungen auf DB**

4. Transaction Tuning
5. Hardware Tuning

**Potentiell hohe Auswirkungen auf DB**

# Das Log ist dein Freund

PARIS LODRON UNIVERSITÄT SALZBURG

- Datenbank Log ist erster Anlaufpunkt
  - u.U. temporär Log-Level erhöhen
- Wir suchen nach
  - Auffälligen Mustern (Anti-Patterns)
  - Anfragen mit besonders hoher Laufzeit
  - Ähnliche Anfragen die hohen Lastanteil besitzen

# Datenbank Tuning

SQL Anti-Patterns finden und entfernen

# SQL Anti-Patterns

PARIS LODRON UNIVERSITÄT SALZBURG

- Beispiel aus dem LOG

| # | Statements | Result |
|---|---|---|
| 1 | `SELECT E.empId FROM Employees E WHERE E.department = 'sales'` | 12 |
| 2 | `SELECT E.name, E.surname FROM Employees E WHERE E.id = 12` | John, Doe |
| 3 | `SELECT E.birthday, E.phone FROM Employees E WHERE E.id = 12` | 12.03.1985, 01259863448 |
| 4 | `SELECT count(orders) FROM Orders OWHERE O.empId = 12` | 36 |

- Analoges Muster existiert häufig

# SQL Anti-Patterns

PARIS LODRON UNIVERSITÄT SALZBURG

- Beispiel aus dem LOG

| # | Statements | Result |
|---|---|---|
| 1 | `SELECT E.empId FROM Employees E WHERE E.department = 'sales'` | 12 |
| 2 | `SELECT E.name, E.surname FROM Employees E WHERE E.id = 12` | John, Doe |
| 3 | `SELECT E.birthday, E.phone FROM Employees E WHERE E.id = 12` | 12.03.1985, 01259863448 |
| 4 | `SELECT count(orders) FROM Orders OWHERE O.empId = 12` | 36 |

- Was ist hier passiert?
- Warum hat das negative Auswirkungen?

# SQL Anti-Patterns

PARIS LODRON UNIVERSITÄT SALZBURG

- Anwendung hat Funktion zum Bestimmen unserer besten Verkäufer

```
Liste<Verkäufer> l = bestimmeMitarbeiter(„sales“)
For each Verkäufer v in l
    print(v.getName())
    print(v.contactData())
    print(v.countOrders())
```

- Anwendungsentwickler sieht DB-Zugriff nicht
  - Versteckt hinter API oder OR-Mapper (Hibernate)
  - Jede Funktion setze einzelnes SQL-Statement ab in dem Teile der Information holt: **langsam**

# SQL Anti-Patterns

PARIS LODRON UNIVERSITÄT SALZBURG

- Anwendung hat Funktion zum Bestimmen unserer besten Verkäufer

```
Liste<Verkäufer> l = bestimmeMitarbeiter(„sales“)
For each Verkäufer v in l
    print(v.getName())
    print(v.contactData())
    print(v.countOrders())
```

```
String getName()
  sql = „Select E.name, E.surname from Employee E “
   + „Where E.id =“ + this.id;
  Result r = exec_sql(sql);
  return r.get(1,„name“)+ „, “ +r.get(1, „surname“);
```

# SQL Anti-Patterns

PARIS LODRON UNIVERSITÄT SALZBURG

- Bruch zwischen Client und Datenbank
  - Andere Datenmodelle: objekt-orientiertes Model vs. relationales Datenmodell
  - Impedance Missmatch
- Konsequenz: Unnötige Arbeit auf DB-Seite

# SQL Anti-Patterns

PARIS LODRON UNIVERSITÄT SALZBURG

- Beispiel aus dem LOG

| # | Statements | Result |
|---|---|---|
| 1 | `SELECT E.empId FROM Employees E WHERE E.department = 'sales'` | 12 |
| 2 | `SELECT E.name, E.surname FROM Employees E WHERE E.id = 12` | John, Doe |
| 3 | `SELECT E.birthday, E.phone FROM Employees E WHERE E.id = 12` | 12.03.1985, 01259863448 |
| 4 | `SELECT count(orders) FROM Orders OWHERE O.empId = 12` | 36 |

**Circuitous Treasure Hunt Anti-Pattern**

# Weitere SQL Anti-Patterns

PARIS LODRON UNIVERSITÄT SALZBURG

- Stifle

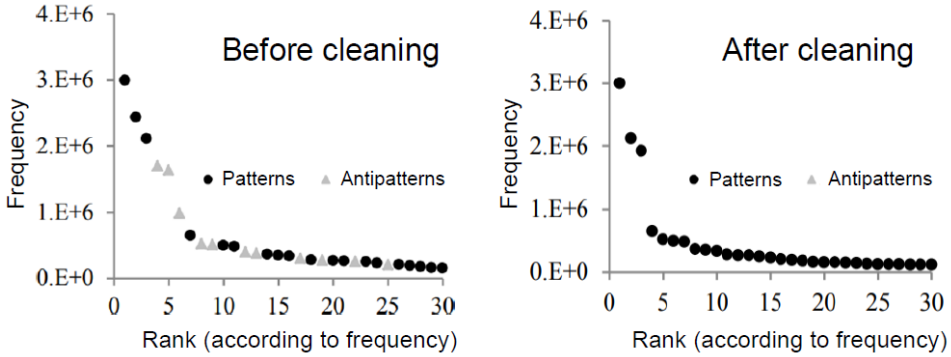
```
Select sum(cost) from R where brand = 1;
Select sum(cost) from R where brand = 2;
Select sum(cost) from R where brand = 3;
Select sum(cost) from R where brand = 4;
Select sum(cost) from R where brand = 5;
```

- Empty semi-truck

```
Client:
get_name(int id)
  Result r = exec_sql(Select * from LargeTable);
  for(Tuple t : r)
    if(t.get(„id“)==id)
      return t.get(name);
  return „not found“;
```

# Lösung

- Anti-Pattern ist ein Muster
- Automatische Suche nach solchen Mustern, dann **Client Re-Factoring**
- Praktische Relevanz: Analyse Log Sloan Digital SkyServer
  - Ca. 25% der Anfragen sind Anti-Patterns




https://ieeexplore.ieee.org/document/8103787

# Take-aways

PARIS LODRON UNIVERSITÄT SALZBURG

- Was sind SQL-Antipatterns?
- Wie entstehen sie?
- Illustration von Beispielen
- Warum lösen wir sie nicht in der Datenbank?